

DATASET CLEANSING CAFE



M. ISLAM AL-WALI 2318076



IMPORT LIBRARIES

```
import pandas as pd
```

```
# Langkah 1: Mount Google Drive  
from google.colab import drive  
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

Pada tahap awal, dilakukan import library Pandas yang digunakan untuk membaca, mengolah, dan membersihkan dataset transaksi penjualan cafe. Selanjutnya, Google Drive dihubungkan dengan Google Colab menggunakan `drive.mount()` agar file dataset yang tersimpan di Google Drive dapat diakses dan digunakan dalam proses data cleansing.



BACA DATASET CAFE YANG KOTOR

```
: # Tampilkan data  
print(data)
```

	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	\
0	TXN_1961373	Coffee	2	2.0	4.0	
1	TXN_4977031	Cake	4	3.0	12.0	
2	TXN_4271903	Cookie	4	1.0	ERROR	
3	TXN_7034554	Salad	2	5.0	10.0	
4	TXN_3160411	Coffee	2	2.0	4.0	
...	...	...	...	...	...	
9995	TXN_7672686	Coffee	2	2.0	4.0	
9996	TXN_9659401	NaN	3	NaN	3.0	
9997	TXN_5255387	Coffee	4	2.0	8.0	
9998	TXN_7695629	Cookie	3	NaN	3.0	
9999	TXN_6170729	Sandwich	3	4.0	12.0	

	Payment Method	Location	Transaction Date
0	Credit Card	Takeaway	2023-09-08
1	Cash	In-store	2023-05-16
2	Credit Card	In-store	2023-07-19
3	UNKNOWN	UNKNOWN	2023-04-27
4	Digital Wallet	In-store	2023-06-11
...	...	...	...
9995	NaN	UNKNOWN	2023-08-30
9996	Digital Wallet	NaN	2023-06-02
9997	Digital Wallet	NaN	2023-03-02
9998	Digital Wallet	NaN	2023-12-02
9999	Cash	In-store	2023-11-07

[10000 rows x 8 columns]

Pada tahap ini, dataset dirty_cafe_sales.csv yang tersimpan di Google Drive dibaca menggunakan fungsi pd.read_csv(). Setelah berhasil dibaca, data ditampilkan untuk melihat isi dan struktur awal dataset. Dataset terdiri dari 10.000 baris dan 8 kolom, yaitu Transaction ID, Item, Quantity, Price Per Unit, Total Spent, Payment Method, Location, dan Transaction Date. Dari tampilan awal juga dapat ditemukan beberapa data yang belum sesuai, seperti nilai NaN, ERROR, dan UNKNOWN, sehingga dataset perlu melalui proses data cleansing sebelum digunakan untuk analisis lebih lanjut.

MENAMPILKAN STRUKTUR VARIABLE DATA CAFE

```
print("Struktur Data Cafe:")
print(data.info())
```

Struktur Data Cafe:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Transaction ID         10000 non-null object
1   Item                   9667 non-null  object
2   Quantity               9862 non-null  object
3   Price Per Unit         9821 non-null  object
4   Total Spent            10000 non-null float64
5   Payment Method         7421 non-null  object
6   Location                6735 non-null  object
7   Transaction Date       9841 non-null  object
dtypes: float64(1), object(7)
memory usage: 625.1+ KB
None
```

```
# Tampilkan nama kolom dan tipe data kolom
column_types = data.dtypes
print(column_types)
```

Transaction ID	object
Item	object
Quantity	object
Price Per Unit	object
Total Spent	object
Payment Method	object
Location	object
Transaction Date	object

dtype: object

Pada tahap ini, fungsi `data.info()` digunakan untuk mengetahui struktur dan kondisi dataset secara keseluruhan. Dataset terdiri dari 10.000 data dengan 8 kolom, di mana sebagian besar kolom masih memiliki tipe data `object`. Hasil pemeriksaan juga menunjukkan adanya data yang kosong atau `null`, terutama pada kolom `Item`, `Quantity`, `Price Per Unit`, `Payment Method`, `Location`, dan `Transaction Date`. Informasi ini digunakan sebagai dasar untuk menentukan data yang perlu diperbaiki atau dibersihkan pada tahap selanjutnya.

DATA CLEANSING-STANDARISASI KOLOM KODE CAFE

```
: # Function to format 'Transaction ID'
def format_transaction_id(transaction_id):
    if isinstance(transaction_id, str):
        if transaction_id.startswith("TXN_"):
            angka = transaction_id.split('_')[-1]
            if angka.isdigit():
                return "TXN_" + angka.zfill(7)
        return transaction_id

: # Apply the function to the 'Transaction ID' column
data['Transaction ID'] = data['Transaction ID'].apply(format_transaction_id)

: # Display the updated 'Transaction ID' column
data[['Transaction ID']].head()
```

	Transaction ID
0	TXN_1961373
1	TXN_4977031
2	TXN_4271903
3	TXN_7034554
4	TXN_3160411

Pada tahap ini dilakukan standarisasi pada kolom Transaction ID untuk memastikan setiap ID transaksi memiliki format yang konsisten. Fungsi `format_transaction_id()` memeriksa apakah ID diawali dengan `TXN_` dan bagian angkanya berupa angka, kemudian memastikan jumlah digitnya terdiri dari 7 digit menggunakan `zfill()`. Setelah fungsi diterapkan, format Transaction ID menjadi lebih seragam sehingga dapat digunakan dengan lebih baik dalam proses pengolahan dan analisis data.

DATA CLEANSING-STANDARISASI KOLOM TRANSACTION ID

```
import pandas as pd
import re

# Baca file .csv
file_path = '/content/drive/MyDrive/Colab Notebooks/DatasetCafe/dirty_cafe_sales.csv'
df = pd.read_csv(file_path)

# Define a function to clean the 'Transaction ID' column
def clean_transaction_id(transaction_id):
    # Define patterns to remove
    patterns = [
        r'\s+',          # Menghapus spasi
        r'^A-Z0-9_',     # Menghapus karakter selain huruf, angka, dan _
    ]

    # Compile the patterns into a single regex pattern
    combined_pattern = re.compile('|'.join(patterns), flags=re.IGNORECASE)

    # Remove the patterns
    cleaned_transaction_id = combined_pattern.sub('', str(transaction_id)).strip().upper()

    return cleaned_transaction_id

# Apply the cleaning function to the 'Transaction ID' column
df['Transaction ID Bersih'] = df['Transaction ID'].apply(clean_transaction_id)

# Display the cleaned DataFrame
df[['Transaction ID', 'Transaction ID Bersih']].head()
```

	Transaction ID	Transaction ID Bersih
0	TXN_1961373	TXN_1961373
1	TXN_4977031	TXN_4977031
2	TXN_4271903	TXN_4271903
3	TXN_7034554	TXN_7034554
4	TXN_3160411	TXN_3160411

Pada tahap ini dilakukan proses pembersihan pada kolom Transaction ID menggunakan library re (Regular Expression). Proses ini bertujuan untuk menghapus spasi dan karakter yang tidak sesuai dengan format ID transaksi, kemudian mengubah huruf menjadi huruf kapital. Hasil pembersihan disimpan pada kolom baru Transaction ID Bersih. Berdasarkan hasil yang ditampilkan, nilai Transaction ID sebelum dan sesudah proses cleansing tetap sama karena data pada bagian tersebut sudah memiliki format yang sesuai.

DATA CLEANSING-STANDARISASI KOLOM TRANSACTION DATE

```
# Define a function to standardize the date format
def standardize_date(date):
    try:
        # Convert the date to datetime format and then to the desired format
        standardized_date = pd.to_datetime(date, errors='coerce').strftime('%d-%m-%Y')
    except:
        standardized_date = None
    return standardized_date

# Apply the standardization function to the 'Transaction Date' column
df['Transaction Date Standar'] = df['Transaction Date'].apply(standardize_date)

# Display the DataFrame with the standardized dates
df[['Transaction Date', 'Transaction Date Standar']].head()
```

	Transaction Date	Transaction Date Standar
0	2023-09-08	08-09-2023
1	2023-05-16	16-05-2023
2	2023-07-19	19-07-2023
3	2023-04-27	27-04-2023
4	2023-06-11	11-06-2023

Pada tahap ini dilakukan standarisasi pada kolom Transaction Date agar seluruh tanggal memiliki format yang seragam. Data tanggal dikonversi ke format tanggal menggunakan `pd.to_datetime()`, kemudian diubah menjadi format DD-MM-YYYY. Hasil standarisasi disimpan pada kolom baru Transaction Date Standar. Contohnya, tanggal 2023-09-08 diubah menjadi 08-09-2023, sehingga data tanggal menjadi lebih konsisten dan mudah dibaca.

MENDETEKSI DATA DUPLIKAT

<pre># Find duplicate rows based on columns 'Item', 'Quantity', 'Price Per Unit', # 'Total Spent', 'Payment Method', 'Location', and 'Transaction Date' duplicate_rows = df[df.duplicated(['Item', 'Quantity', 'Price Per Unit', 'Total Spent', 'Payment Method', 'Location', 'Transaction Date'], keep=False)] # Display the duplicate rows duplicate_rows</pre>										
	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date	Transaction ID Bersih	Transaction Date Standar
5	TXN_2602893	Smoothie	5	4.0	20.0	Credit Card	NaN	2023-03-31	TXN_2602893	31-03-2023
27	TXN_5695074	Juice	4	3.0	12.0	Credit Card	Takeaway	2023-04-10	TXN_5695074	10-04-2023
115	TXN_1001832	Salad	2	5.0	10.0	Cash	Takeaway	UNKNOWN	TXN_1001832	None
126	TXN_9646452	Juice	5	3.0	15.0	Credit Card	NaN	2023-09-16	TXN_9646452	16-09-2023
300	TXN_1623966	Tea	2	1.5	3.0	Cash	Takeaway	UNKNOWN	TXN_1623966	None
...	...	...	...	...	...	...	...	...	...	...
9898	TXN_8999360	Juice	3	3.0	9.0	Cash	Takeaway	2023-09-27	TXN_8999360	27-09-2023
9909	TXN_2112522	Sandwich	5	4.0	20.0	Digital Wallet	Takeaway	2023-04-30	TXN_2112522	30-04-2023
9930	TXN_4428252	Sandwich	3	4.0	12.0	NaN	Takeaway	2023-10-31	TXN_4428252	31-10-2023
9967	TXN_8563793	Juice	4	3.0	12.0	NaN	In-store	2023-03-26	TXN_8563793	26-03-2023
9989	TXN_1741685	Juice	5	3.0	15.0	Cash	NaN	2023-08-18	TXN_1741685	18-08-2023

Pada tahap ini dilakukan pemeriksaan terhadap data duplikat berdasarkan beberapa kolom yang menggambarkan detail transaksi, yaitu Item, Quantity, Price Per Unit, Total Spent, Payment Method, Location, dan Transaction Date. Fungsi duplicated() digunakan untuk menemukan baris yang memiliki kombinasi data yang sama. Hasil pemeriksaan menunjukkan adanya beberapa transaksi yang memiliki data serupa, sehingga data tersebut perlu diperiksa dan ditangani agar tidak memengaruhi hasil analisis pada tahap selanjutnya.

DATA ENRICHMENT

```
# Count the missing values in the 'Total Spent' column
missing_values_count = df['Total Spent'].isna().sum()
missing_values_count
```

```
np.int64(173)
```

```
# Ubah kolom 'Total Spent' menjadi tipe numerik terlebih dahulu, ubah nilai non-angka (seperti 'ERROR') menjadi NaN
data['Total Spent'] = pd.to_numeric(data['Total Spent'], errors='coerce')
```

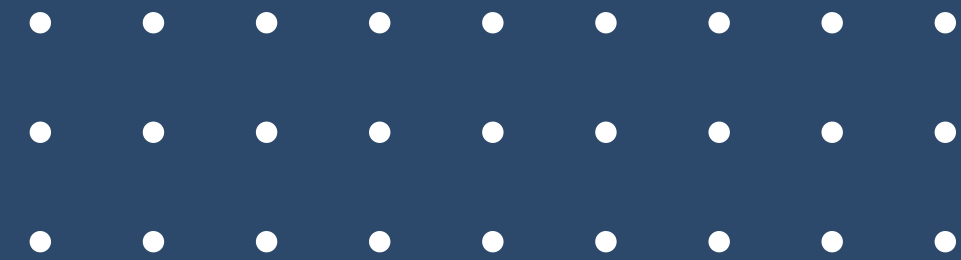
```
# Menghitung rata-rata dari kolom 'Total Spent', abaikan nilai yang hilang (NA)
mean_value = data['Total Spent'].mean()
```

```
# Mengisi nilai yang hilang dengan langsung menetapkan hasilnya ke kolom (menghindari FutureWarning)
data['Total Spent'] = data['Total Spent'].fillna(mean_value)
```

```
# Tampilkan beberapa baris pertama untuk memastikan hasilnya
data[['Total Spent']].head()
```

	Total Spent
0	4.000000
1	12.000000
2	8.924352
3	10.000000
4	4.000000

Pada tahap ini dilakukan pemeriksaan dan penanganan nilai yang hilang pada kolom Total Spent. Hasil pemeriksaan awal menunjukkan terdapat 173 nilai yang kosong. Selain itu, terdapat nilai non-angka seperti ERROR, sehingga kolom terlebih dahulu dikonversi menjadi tipe numerik menggunakan `pd.to_numeric()`, dengan nilai yang tidak dapat dikonversi diubah menjadi NaN. Selanjutnya, nilai yang hilang diisi menggunakan nilai rata-rata (mean) dari kolom Total Spent. Setelah proses selesai, data pada kolom tersebut menjadi numerik dan tidak lagi memiliki nilai kosong.



TERIMA KASIH

